Project Isabella: Architectural Breakdown

Isabella is an implementation of an **Open-LLM-VTuber** stack, meaning she is built from a constellation of local microservices and AI models working together in real time over WebSockets.

High-Level Flow

```
[ Frontend: Electron + React ] <== Localhost WebSocket ==> [ Backend: Python FastAPI ]

                    (Mic -> VAD) > (Audio Player / Live2D Engine)
                                          ^
                                          |
                                          v
    (ASR: Whisper) -> (Brain: Llama 3.3 70B via OpenRouter) <-> (Tools: MCP Desktop
                                       Controller)
                                          v
                                   (TTS: Kokoro-82M)
```

1. The Frontend (The Body & Sensors)

The frontend is built on **Electron**, which wraps a **React 18** web application (bundled by Vite) into a native Windows desktop window.

- **Ears (Voice Activity Detection):** It uses `@ricky0123/vad-web` running directly in the browser environment. It listens to your microphone constantly, but uses a local neural network to detect *when you actually speak*. When you pause, it slices that audio chunk and shoots the raw Float32Array audio buffer over the WebSocket to the backend.
- **Body (Live2D Engine):** It uses `pixi.js` combined with `pixi-live2d-display` and the official Cubism Core SDK. This allows the 2D "Shizuku" model to be hardware-accelerated.
- **Lip Sync & Emotion:** When the backend sends back voice audio, the frontend analyzes the audio frequencies to make the avatar's lips move dynamically. When it receives tags like `[joy]` or `[sadness]`, it triggers pre-baked motion files in the Live2D model.

2. The Backend (The Nervous System)

The backend is a **Python FastAPI** server utilizing `uvicorn`. It acts as the central hub, completely isolated from the UI, processing data as fast as possible.

- **Transcription (ASR - Automatic Speech Recognition):** You are configured to use **faster_whisper**. When the backend receives your raw audio buffer, Faster-Whisper uses local CPU/GPU compute to instantly transcribe your voice into text.
- **The Brain (Agentic Loop):** The backend uses an Agent class. It holds Isabella's system prompt (her personality, memories, and rules) and appends your transcribed text. It then reaches out to **OpenRouter** (using the `meta-llama/llama-3.3-70b-instruct` model) to generate her response.
- **Voice Synthesis (TTS - Text-to-Speech):** As OpenRouter streams text back word-by-word, the backend pipes those words into **kokoro_tts** (Kokoro-82M). Kokoro turns the text into .wav audio bytes on the fly and streams them back down the WebSocket to the frontend.
- **The Hands (MCP Desktop Controller):** This is the custom `desktop_mcp.py` script. The LLM is given JSON schemas of functions it can run. If you ask Isabella to play a song, the LLM outputs a JSON command instead of text. The backend intercepts this, runs your Python functions to control the PC, and feeds the result back to the LLM so she can tell you she did it.

The Full Loop (What happens in 2 seconds)

1. You say "Open Notepad and say hi."
2. **VAD** detects you stopped speaking and sends the audio over WebSocket.
3. **Faster-Whisper** transcribes it to text.
4. **Agent** sends it to **Llama 3.3 70B** on OpenRouter.
5. Llama realizes it needs a tool, and fires the `write_to_notepad` function.
6. The Backend executes the Python script. Notepad opens and "hi" is typed.
7. Backend tells Llama it succeeded.
8. Llama generates: `[joy] Right away, Master Sayantan! It is done.`
9. **Kokoro TTS** instantly converts that to a female voice audio file.

10. The Frontend plays the audio, triggers the "joy" animation on Shizuku, and moves her mouth to the voice.